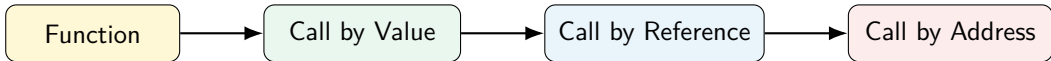


Object Oriented Programming System

Course Code: 25CS301

Lecture 3

Functions and Parameter Passing in C++



B.Tech III Semester

Suggested Duration: 60 Minutes

Lecture Roadmap

- ① Lecture Positioning
- ② Why Functions?
- ③ Function Basics
- ④ Function Declaration, Definition and Call
- ⑤ Parameter Passing
- ⑥ Call by Value
- ⑦ Call by Reference
- ⑧ Call by Address
- ⑨ Comparison
- ⑩ Functions and Classes
- ⑪ Worked Programs
- ⑫ Common Mistakes
- ⑬ Assessment
- ⑭ Summary

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Where Lecture 3 Fits in Unit 1

Previous Lecture

We studied the structure of a C++ program, class, object, data members, member functions, dot operator and scope resolution operator.

Today's Lecture

We study functions and how data is passed to functions.

Why This Matters for OOP

Member functions inside classes are still functions. If students do not understand ordinary functions, they will struggle with member functions, constructors, operator overloading and templates.

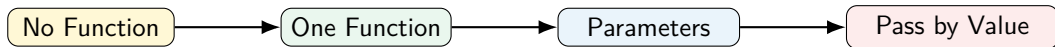
Learning Objectives

By the end of this lecture, students will be able to:

- explain the need for functions in C++ programs;
- write function declaration, definition and function call;
- identify actual and formal parameters;
- compare call by value, call by reference and call by address;
- predict output using dry run and memory diagrams;
- choose the correct parameter passing technique for a problem;
- connect ordinary functions with member functions in OOP.

Teaching Strategy for Beginners

Do not begin with a long student-record program. That is too much cognitive load. Start with one function, one variable, one parameter and one output. Then increase complexity slowly.



Lecture Roadmap

- 1 Lecture Positioning
- 2 **Why Functions?**
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Problem Without Functions

Suppose a program needs to calculate the square of many numbers.

Without functions, the same logic is repeated again and again.

- Repeated code increases program size.
- Debugging becomes difficult.
- Updating logic becomes risky.
- Code becomes less readable.

A function is introduced when a task has a clear purpose and may be reused.

Repeated Code Example

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int a = 5;
7     int b = 7;
8     int c = 9;
9
10    cout << a * a << endl;
11    cout << b * b << endl;
12    cout << c * c << endl;
13
14    return 0;
15 }
```

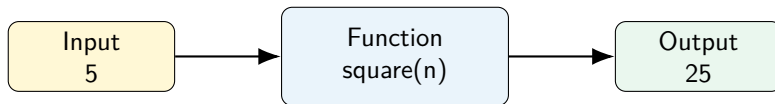
This code is correct, but it is not well-designed. Correct output does not always mean good programming style

Better Version Using Function

```
1 #include <iostream>
2 using namespace std;
3 int square(int n) {
4     return n * n;
5 }
6 int main() {
7     cout << square(5) << endl;
8     cout << square(7) << endl;
9     cout << square(9) << endl;
10    return 0;
11 }
```

The logic of squaring is written once and reused many times.

Function as a Machine



Simple Idea

A function accepts input, performs a task and may return output.

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics**
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

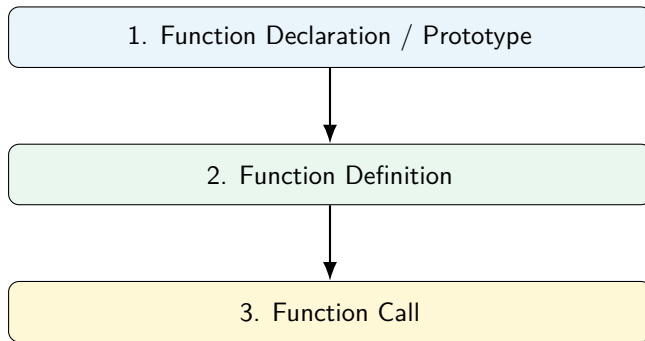
Definition of Function

Function

A function is a named block of statements that performs a specific task and can be executed whenever required by calling its name.

- It improves modularity.
- It reduces repetition.
- It makes testing easier.
- It supports top-down program design.
- It prepares students for member functions in classes.

Three Parts of a Function



Function Syntax

```
1 return_type function_name(parameter_list)
2 {
3     // body of function
4     return value;
5 }
```

Parts

Part	Meaning
return type	Type of value returned by function
function name	Name used to call the function
parameter list	Values received by function
function body	Statements executed by function
return statement	Sends result back to caller

Smallest Useful Function

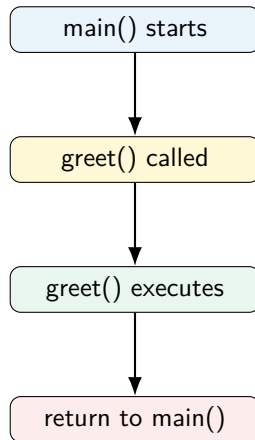
```
1 #include <iostream>
2 using namespace std;
3
4 void greet()
5 {
6     cout << "Welcome to OOPS with C++" << endl;
7 }
8
9 int main()
10 {
11     greet();
12     return 0;
13 }
```

A void function performs a task but does not return a value.

Dry Run: `greet()` Function

- 1 Program execution starts from `main()`.
- 2 Statement `greet()`; is encountered.
- 3 Control jumps to the function `greet()`.
- 4 Message is printed.
- 5 Function ends.
- 6 Control returns to `main()`.
- 7 Program terminates after `return 0`;

Control Flow Diagram



Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call**
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Declaration Before main()

```
1 #include <iostream>
2 using namespace std;
3
4 void greet();           // function declaration
5
6 int main()
7 {
8     greet();           // function call
9     return 0;
10 }
11
12 void greet()           // function definition
13 {
14     cout << "Hello Students" << endl;
15 }
```

Why Declaration is Needed

Compiler Rule

The compiler reads a C++ program mostly from top to bottom. Before a function is called, the compiler must know its name, return type and parameter list.

Function declaration tells the compiler: “This function exists. Its full body may appear later.”

Declaration vs Definition vs Call

Concept	Meaning	Example
Declaration	Announces function to compiler	<code>int add(int,int);</code>
Definition	Contains actual body	<code>int add(...) {...}</code>
Call	Executes the function	<code>add(5,7);</code>

Students often write function definition but forget to call it. A function does nothing until it is called.

Function Returning a Value

```
1 #include <iostream>
2 using namespace std;
3
4 int add(int a, int b)
5 {
6     int sum = a + b;
7     return sum;
8 }
9
10 int main()
11 {
12     int result = add(10, 20);
13     cout << "Sum = " << result;
14     return 0;
15 }
```

Actual and Formal Parameters

Formal Parameters

Variables written in the function definition or declaration.

Actual Parameters / Arguments

Values passed during the function call.

In `int result = add(10, 20);`, values 10 and 20 are actual parameters. In `int add(int a, int b)`, variables a and b are formal parameters.

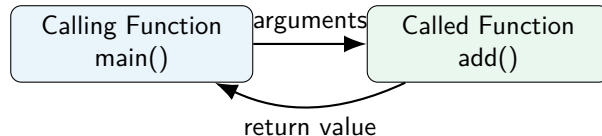
Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing**
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

What is Parameter Passing?

Parameter Passing

Parameter passing is the mechanism through which data is sent from the calling function to the called function.



Types Covered Today

- ① Call by Value
- ② Call by Reference
- ③ Call by Address

These three techniques are essential for understanding functions, arrays, pointers, references and object manipulation in later lectures.

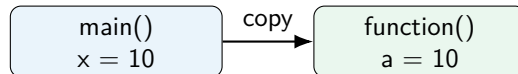
Lecture Roadmap

- ① Lecture Positioning
- ② Why Functions?
- ③ Function Basics
- ④ Function Declaration, Definition and Call
- ⑤ Parameter Passing
- ⑥ Call by Value**
- ⑦ Call by Reference
- ⑧ Call by Address
- ⑨ Comparison
- ⑩ Functions and Classes
- ⑪ Worked Programs
- ⑫ Common Mistakes
- ⑬ Assessment
- ⑭ Summary

Call by Value: Concept

Definition

In call by value, a copy of the actual argument is passed to the function. Changes made inside the function do not affect the original variable.



Original variable is safe because the function works on a copy.

Call by Value Example

```
1 #include <iostream>
2 using namespace std;
3
4 void change(int a)
5 {
6     a = 50;
7     cout << "Inside function: " << a << endl;
8 }
9
10 int main()
11 {
12     int x = 10;
13     change(x);
14     cout << "Inside main: " << x << endl;
15     return 0;
16 }
```

Output and Explanation

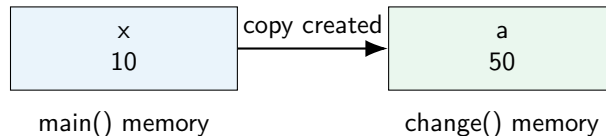
Output

```
Inside function: 50
```

```
Inside main: 10
```

- x contains 10 in main().
- A copy of x is created as a.
- a changes to 50.
- x remains 10.

Memory View: Call by Value



Do not say the function changed `x`. It changed only its local copy `a`.

When to Use Call by Value

Use call by value when:

- the function only needs to read the value;
- the original data must not be modified;
- data size is small, such as `int`, `char`, `float`;
- safety is more important than modification.

Functions such as `square(n)`, `isEven(n)` and `max(a,b)` commonly use call by value.

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference**
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Call by Reference: Concept

Definition

In call by reference, the function parameter becomes an alias of the actual argument. No copy is created. Changes made inside the function affect the original variable.

Reference means another name for the same memory location.

Reference Variable Quick Reminder

```
1 int x = 10;  
2 int &r = x;  
3  
4 r = 25;  
5 cout << x;    // prints 25
```

Meaning

`r` is not a new variable. It is another name for `x`.

Call by Reference Example

```
1 #include <iostream>
2 using namespace std;
3
4 void change(int &a)
5 {
6     a = 50;
7     cout << "Inside function: " << a << endl;
8 }
9
10 int main()
11 {
12     int x = 10;
13     change(x);
14     cout << "Inside main: " << x << endl;
15     return 0;
16 }
```

Output and Explanation

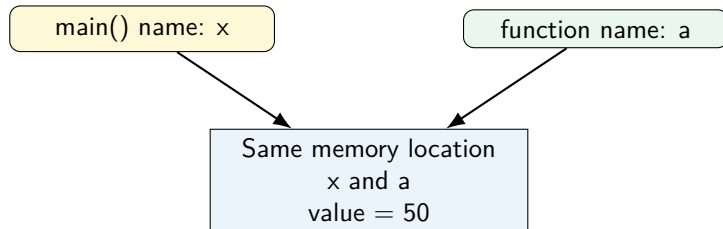
Output

```
Inside function: 50
```

```
Inside main: 50
```

- `x` is passed to `change()`.
- Parameter `a` becomes an alias of `x`.
- Changing `a` changes `x`.

Memory View: Call by Reference



When to Use Call by Reference

Use call by reference when:

- the function must modify the original variable;
- copying is expensive;
- multiple values need to be returned through parameters;
- you want cleaner syntax than pointers.

Swap function is the best first example for call by reference.

Swap Using Call by Reference

```
1 #include <iostream>
2 using namespace std;
3
4 void swapNumbers(int &a, int &b)
5 {
6     int temp = a;
7     a = b;
8     b = temp;
9 }
10
11 int main()
12 {
13     int x = 10, y = 20;
14     swapNumbers(x, y);
15     cout << x << " " << y;
16     return 0;
17 }
```

Dry Run: swapNumbers(x,y)

Step	temp	x / a	y / b
Initial	-	10	20
temp = a	10	10	20
a = b	10	20	20
b = temp	10	20	10
Final	10	20	10

The values in x and y are actually swapped because references are used.

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address**
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Call by Address: Concept

Definition

In call by address, the address of the actual argument is passed to the function. The function uses pointer variables to access and modify the original data.

Address means memory location. Pointer means variable that stores an address.

Address and Pointer Symbols

Symbol	Meaning
<code>&x</code>	Address of variable <code>x</code>
<code>int *p</code>	Pointer variable that can store address of integer
<code>*p</code>	Value stored at the address held by <code>p</code>

The symbol `*` has different meanings depending on context: pointer declaration and dereferencing.

Call by Address Example

```
1 #include <iostream>
2 using namespace std;
3
4 void change(int *p)
5 {
6     *p = 50;
7     cout << "Inside function: " << *p << endl;
8 }
9
10 int main()
11 {
12     int x = 10;
13     change(&x);
14     cout << "Inside main: " << x << endl;
15     return 0;
16 }
```

Output and Explanation

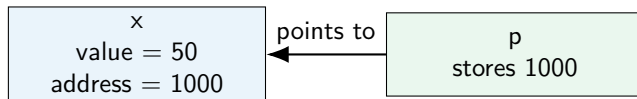
Output

```
Inside function: 50
```

```
Inside main: 50
```

- `&x` passes the address of `x`.
- `p` stores this address.
- `*p = 50` modifies the value stored at that address.
- Therefore `x` becomes 50.

Memory View: Call by Address



Swap Using Call by Address

```
1 #include <iostream>
2 using namespace std;
3
4 void swapNumbers(int *a, int *b)
5 {
6     int temp = *a;
7     *a = *b;
8     *b = temp;
9 }
10
11 int main()
12 {
13     int x = 10, y = 20;
14     swapNumbers(&x, &y);
15     cout << x << " " << y;
16     return 0;
17 }
```

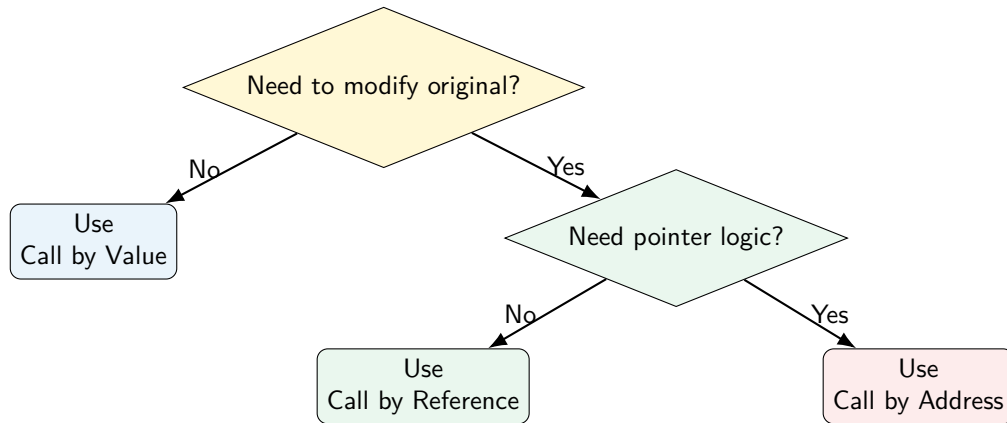
Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison**
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Comparison of Three Techniques

Point	Call by Value	Call by Reference	Call by Address
What is passed?	Copy of value	Alias	Address
Original changed?	No	Yes	Yes
Syntax in call	<code>f(x)</code>	<code>f(x)</code>	<code>f(&x)</code>
Syntax in function	<code>int a</code>	<code>int &a</code>	<code>int *a</code>
Beginner friendly?	Easiest	Moderate	Harder
Risk	Low	Medium	High if misused

Decision Rule



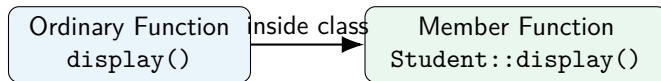
Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes**
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Connection with Classes and Objects

Important Link

A member function is a function that belongs to a class.



Ordinary Function vs Member Function

Ordinary Function

```
1 void display()  
2 {  
3     cout << "Hello";  
4 }
```

Member Function

```
1 class Student  
2 {  
3     public:  
4         void display()  
5         {  
6             cout << "Hello";  
7         }  
8 };
```

Passing Object to a Function: Simple Preview

```
1 class Box
2 {
3 public:
4     int length;
5 };
6
7 void show(Box b)
8 {
9     cout << b.length;
10 }
11
12 int main()
13 {
14     Box b1;
15     b1.length = 10;
16     show(b1);
17 }
```

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs**
- 12 Common Mistakes
- 13 Assessment
- 14 Summary

Program 1: Simple Function

```
1 #include <iostream>
2 using namespace std;
3
4 void welcome()
5 {
6     cout << "Welcome to C++ Functions" << endl;
7 }
8
9 int main()
10 {
11     welcome();
12     return 0;
13 }
```

Program 2: Function with Return Value

```
1 #include <iostream>
2 using namespace std;
3
4 int cube(int n)
5 {
6     return n * n * n;
7 }
8
9 int main()
10 {
11     int number = 3;
12     cout << "Cube = " << cube(number);
13     return 0;
14 }
```

Program 3: Maximum of Two Numbers

```
1 #include <iostream>
2 using namespace std;
3
4 int maximum(int a, int b)
5 {
6     if (a > b)
7         return a;
8     else
9         return b;
10 }
11
12 int main()
13 {
14     cout << maximum(15, 22);
15     return 0;
16 }
```

Program 4: Swap Using Reference

```
1 #include <iostream>
2 using namespace std;
3
4 void swapNumbers(int &a, int &b)
5 {
6     int temp = a;
7     a = b;
8     b = temp;
9 }
10
11 int main()
12 {
13     int x = 4, y = 8;
14     swapNumbers(x, y);
15     cout << x << " " << y;
16     return 0;
17 }
```

Program 5: Swap Using Address

```
1 #include <iostream>
2 using namespace std;
3
4 void swapNumbers(int *a, int *b)
5 {
6     int temp = *a;
7     *a = *b;
8     *b = temp;
9 }
10
11 int main()
12 {
13     int x = 4, y = 8;
14     swapNumbers(&x, &y);
15     cout << x << " " << y;
16     return 0;
17 }
```

Program 6: Function with Class Preview

```
1 #include <iostream>
2 using namespace std;
3 class Chalk {
4 public:
5     string colour;
6 };
7 void showColour(Chalk c) {
8     cout << "Chalk colour = " << c.colour;
9 }
10 int main() {
11     Chalk c1;
12     c1.colour = "White";
13     showColour(c1);
14     return 0;
15 }
```

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes**
- 13 Assessment
- 14 Summary

Common Mistakes in Functions

- Forgetting semicolon after function declaration.
- Writing return value in a void function without purpose.
- Calling function before declaration.
- Confusing actual and formal parameters.
- Expecting call by value to modify original variables.
- Writing `swapNumbers(&x, &y)` for a reference-based function.
- Forgetting `*` while modifying through pointer.

Debug This Code

```
1 #include <iostream>
2 using namespace std;
3
4 int add(int a, int b)
5 {
6     cout << a + b;
7 }
8
9 int main()
10 {
11     int result = add(5, 6);
12     cout << result;
13     return 0;
14 }
```

Question

What is wrong with this function?

Answer: Debug This Code

- Function return type is `int`.
- But the function does not return any integer value.
- It only prints the result.

Correction

Either change return type to `void`, or return `a + b`.

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment**
- 14 Summary

Viva Questions

- 1 What is a function and why are functions used in C++?
- 2 What is the difference between declaration and definition?
- 3 What is a function call?
- 4 What are formal and actual parameters?
- 5 What is call by value?
- 6 Why does call by value not modify the original variable?
- 7 What is call by reference?
- 8 What is call by address?
- 9 What is a pointer?
- 10 What is the use of & in call by address?
- 11 What is the use of * in pointer dereferencing?

MCQ 1

Question

Which parameter passing method creates a copy of the actual argument?

- ☐ A Call by reference
- ☐ B Call by address
- ☐ C Call by value
- ☐ D Call by pointer only

Answer: C

MCQ 2

Question

Which symbol is used to declare a reference variable?

- ☐ A *
- ☐ B &
- ☐ C #
- ☐ D ::

Answer: B

MCQ 3

Question

In call by address, what is passed to the function?

- ☐ A Copy of value
- ☐ B Address of variable
- ☐ C Name of class
- ☐ D Return type

Answer: B

Short Answer Questions

- 1 Explain the need for functions with one example.
- 2 Differentiate between function declaration and function definition.
- 3 Explain call by value with a memory diagram.
- 4 Explain call by reference with an example.
- 5 Compare call by reference and call by address.

Programming Exercises

- 1 Write a function to check whether a number is even or odd.
- 2 Write a function to calculate factorial of a number.
- 3 Write a function to find maximum of three numbers.
- 4 Write a function to swap two numbers using reference.
- 5 Write a function to swap two numbers using address.
- 6 Write a function that accepts radius and returns area of circle.

Assignment

Submission Task

Write a C++ program that accepts marks of three subjects and uses separate functions to calculate total, percentage and grade.

Compulsory Requirement

Use at least three user-defined functions and explain which parameter passing technique you used.

Lecture Roadmap

- 1 Lecture Positioning
- 2 Why Functions?
- 3 Function Basics
- 4 Function Declaration, Definition and Call
- 5 Parameter Passing
- 6 Call by Value
- 7 Call by Reference
- 8 Call by Address
- 9 Comparison
- 10 Functions and Classes
- 11 Worked Programs
- 12 Common Mistakes
- 13 Assessment
- 14 Summary**

One-Slide Summary

- Function is a named block of reusable code.
- Function declaration informs compiler.
- Function definition contains actual logic.
- Function call executes the function.
- Call by value passes copy.
- Call by reference passes alias.
- Call by address passes memory address.
- References are simpler and safer than raw pointers for many beginner-level programs.

Final Memory Trick

Technique	Keyword Idea	Original Changed?
Value	Copy	No
Reference	Alias	Yes
Address	Pointer	Yes

Value protects. Reference modifies simply. Address modifies through pointer.

Next Lecture Preview

Lecture 4

Inline functions, function overloading and default arguments.

- Why small functions may be made inline.
- How same function name can perform different tasks.
- How default values simplify function calls.

End of Lecture 3